IT Workshop - 3

Project Report

Project Details	**Information**
Student Name	Anay Duggal
Roll Number	2025BCS0093
Project	CNN Digit Recognizer
Dataset	MNIST handwritten digits

Abstract

This project is about building a system that can recognise handwritten digits. A Convolutional Neural Network (CNN) is trained using the MNIST dataset, which contains images of digits from 0 to 9. After training, the model is saved and connected to a small Streamlit web application. The application lets a user draw a digit on the screen and then shows the predicted digit, the model's confidence, and a chart of the probabilities for all ten digits. The project also uses training graphs and a confusion matrix to understand how well the model performs.

Introduction

Handwritten digit recognition is a common image-classification problem. Even though there are only ten possible digits, people write them in different ways. Digits can have different shapes, sizes, slants and stroke thicknesses, which makes recognition challenging.

A CNN is useful for this task because it learns patterns from images. During training, it learns features such as edges, curves and combinations of strokes. The goal of this project was to train a CNN on handwritten digit images and then make it usable through a simple web interface.

Objectives

1. Use the MNIST dataset to train a handwritten digit classifier.
2. Prepare the images in a format suitable for a CNN.
3. Build and train a CNN that predicts the digit class.
4. Check the model using test data, learning curves and a confusion matrix.
5. Create a Streamlit application where a user can draw a digit and get a prediction.

Tools and Technologies Used

Tool or Library	How it is used
Python	Used to write the training code and application.
TensorFlow and Keras	Used to build, train, evaluate and save the CNN model.
NumPy	Used for working with image arrays and model predictions.
Matplotlib and Seaborn	Used to display graphs and the confusion matrix.
scikit-learn	Used for the classification report and confusion matrix.
Streamlit	Used to create the web application.
streamlit-drawable-canvas	Provides the area where users draw digits.
Pillow (PIL)	Used to convert and resize the drawn image.
Jupyter Notebook	Used to develop and test the model.

Dataset

The project uses the MNIST dataset, which contains grayscale images of handwritten digits. Each image is 28 by 28 pixels and belongs to one of ten classes, from 0 to 9. The standard MNIST dataset available through Keras has 60,000 training images and 10,000 test images. Training images are used to learn the model's parameters, while the test images are kept for checking the model after training.

Each image has one grayscale channel. Before it is passed to the CNN, its pixel values are converted to floating-point numbers and scaled from the range 0–255 to the range 0–1. The image data is also reshaped so that it has the dimensions expected by the convolutional layers.

Image Preprocessing

1. Load the MNIST training and test data.
2. Convert pixel values to float32 and divide them by 255 to normalise the values.
3. Reshape the images to include the grayscale channel, giving each image a shape of 28 × 28 × 1.
4. Use the digit labels as integer class values from 0 to 9.

CNN Model

Layer	Configuration	Purpose
Input	28 × 28 × 1	Receives the grayscale image.
Convolution	32 filters, 3 × 3, ReLU	Learns simple image patterns.
Max pooling	2 × 2	Reduces the spatial dimensions.
Convolution	64 filters, 3 × 3, ReLU	Learns more detailed patterns.
Max pooling	2 × 2	Reduces the feature maps further.
Flatten	One-dimensional output	Changes the feature maps into a vector.
Dropout	30%	Helps reduce overfitting during training.
Dense	64 neurons, ReLU	Combines the features learned by earlier layers.
Output	10 neurons, Softmax	Produces a probability for each digit from 0 to 9.

Model Training

The model is trained for 10 epochs with a batch size of 64. A validation split of 10% is used to check performance on data that is not used for updating the model's weights in each training step. The training history records loss and accuracy for both the training and validation data.

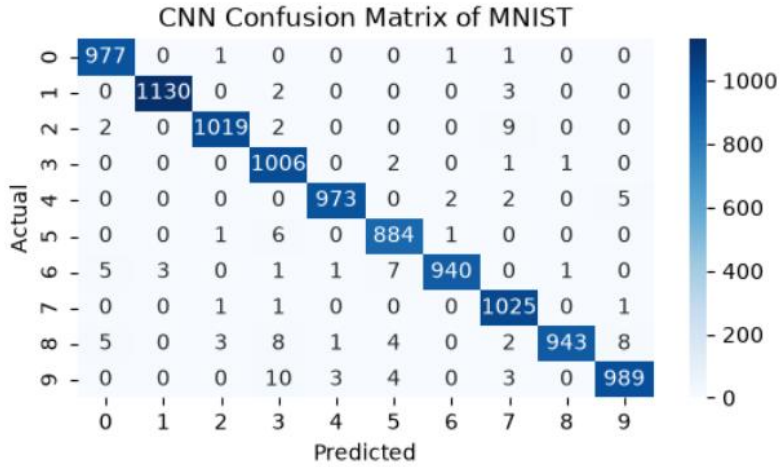
In the supplied accuracy graph, training accuracy rises from about 93.6% in the first epoch to around 99.4% by the last epoch. Validation accuracy stays around 98–99% and finishes at approximately 99.1%. Training loss keeps decreasing. Validation loss decreases at first and then rises slightly near

the end. This can be a sign of mild overfitting, so it would be useful to keep checking validation loss in future experiments.

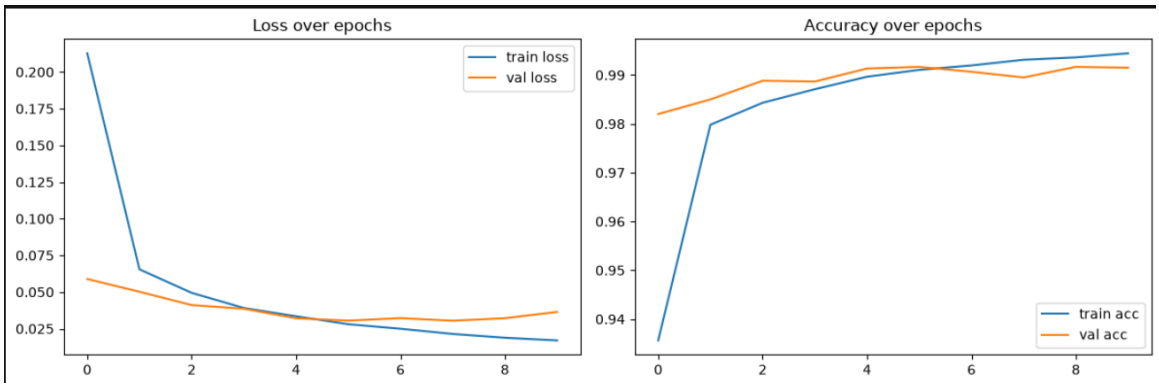
Results and Evaluation

After training, the model is evaluated using the test images. It produces a probability for each digit, and the digit with the highest probability is selected as the prediction. The classification report can be used to examine precision, recall and F1-score for each class. The confusion matrix provides a visual summary of correct and incorrect predictions.

Confusion Matrix



Training and Validation Graphs



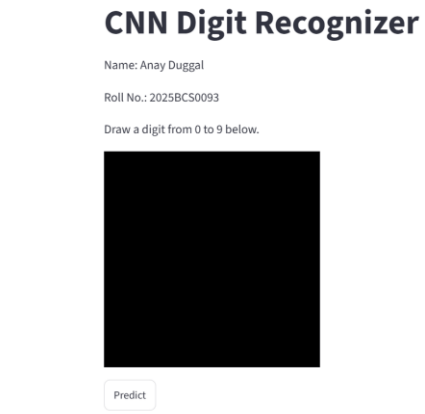
Streamlit Application

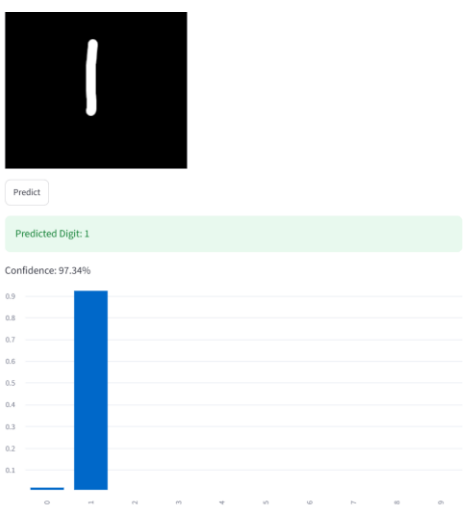
The interactive part of the project is written in r.py. It loads the saved model from model.keras and displays the project title, student details and a drawing canvas. The canvas has a black background and uses a white stroke, so the user can draw a digit in a style similar to the training images.

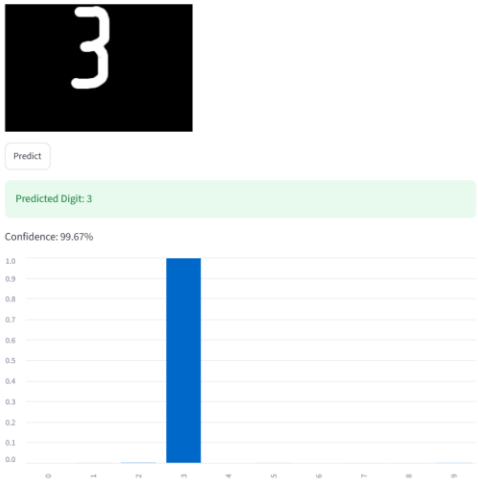
When the user clicks the Predict button, the canvas image is converted into a PIL image. The application changes it to grayscale, resizes it to 28 × 28 pixels, normalises its pixel values and reshapes it to the input format expected by the model. The model then predicts the digit. The

application displays the predicted digit, the confidence percentage and a bar chart showing the probabilities for digits 0 to 9.

Example Predictions







Conclusion

This project combines a CNN model with a simple interactive application to recognise handwritten digits. The model is trained using the MNIST dataset, evaluated with test data and visualised using learning curves and a confusion matrix. The Streamlit interface makes it possible to draw a digit and see the model's prediction without running the notebook each time. The supplied results indicate strong performance on the evaluation run. The project could be improved further by refining the preprocessing of user drawings and reporting the exact test metrics from the notebook.

References

MNIST handwritten digit dataset, available through keras.datasets.mnist.